

ЛАБОРАТОРНА РОБОТА №8

КЛАСИ. Ч. 2

Мета роботи: ознайомитися з ООП в мові Python

Хід роботи:

Завдання 1. Напишіть клас Bank для опису простих операції з вашим банківським рахунком: покласти на рахунок, зняти з рахунку, переглянути рахунок. При створенні екземпляру класу, екземпляр отримує атрибут `__balance` з певним значенням. Клас повинен містити методи для додавання коштів на рахунок і знімання з рахунку, за умови, що на рахунку достатньо коштів.

Лістинг коду:

```
class Bank:
    def __init__(self, balance):
        self._balance = balance

    def deposit(self, amount):
        self._balance += amount

    def withdraw(self, amount):
        if amount <= self._balance:
            self._balance -= amount
        else:
            print("Недостатньо коштів")

    def get_balance(self):
        return self._balance

if __name__ == "__main__":
    account = Bank(1000)
    account.deposit(500)
    account.withdraw(200)
    print(f"Поточний баланс: {account.get_balance()}")
```

Результат виконання:

```
:/study/python_labs/lab-8/task1.py
Поточний баланс: 1300
```

					ДУ «Житомирська політехніка».25.F2.02.000 – Лр8			
Змн.	Арк.	№ докум.	Підпис	Дата				
Розроб.		Гришкова Е. А.			Звіт з лабораторної роботи		Літ.	Арк.
Перевір.		Желізко В. В.						1
Керівник								12
Н. контр.							ФІКТ Гр. ВТк-25-1	
Зав. каф.								

У межах завдання реалізовано клас Bank для моделювання базових фінансових операцій. Атрибут `_balance` визначено як захищений (protected), що є стандартною практикою інкапсуляції в Python. Методи класу забезпечують зміну стану об'єкта: `deposit` виконує інкремент балансу, а `withdraw` містить логічну перевірку на достатність ресурсів перед проведенням транзакції.

Завдання 2. Напишіть клас `Coin`, який описує монету, яку можна підкидати. При створенні екземпляру класу, екземпляр отримує атрибут `__sideup` зі значенням `heads` або `tails`. У класі визначте метод `toss`, який випадково визначає результат підкидання монети - орел чи решка. Створіть екземпляр класу і виведіть на екран `n` підкидань монети.

Лістинг коду:

```
import random

class Coin:
    def __init__(self):
        self._sideup = random.choice(['heads', 'tails'])

    def toss(self):
        self._sideup = random.choice(['heads', 'tails'])

    def get_sideup(self):
        return self._sideup

if __name__ == "__main__":
    my_coin = Coin()
    n = 5
    for i in range(n):
        my_coin.toss()
        print(f"Підкидання {i+1}: {my_coin.get_sideup()}")
```

Результат виконання:

```
Підкидання 1: heads
Підкидання 2: tails
Підкидання 3: tails
Підкидання 4: heads
Підкидання 5: tails
```

Клас `Coin` моделює стохастичний процес підкидання монети. Ініціалізація та метод `toss` базуються на використанні функції `random.choice`, що забезпечує

		Гришкова Е.А.			ДУ «Житомирська політехніка». 25.F2.02.000 – Лр8	Арк.
		Желізько В.В.				2
Змн.	Арк.	№ докум.	Підпис	Дата		

рівноймовірний розподіл між двома станами: 'heads' та 'tails'. Програма демонструє циклічний виклик методу для отримання серії незалежних випробувань.

Завдання 3. Напишіть клас Car, який надає для створених екземплярів такі атрибути даних автомобіля: марку виготовлення автомобіля, модель автомобіля, рік автомобіля, швидкість (початкове значення 0). Клас також повинен мати наступні методи: accelerate (метод повинен щоразу додавати 5 до значення атрибуту даних про швидкість), brake (метод повинен віднімати 5 від значення атрибута даних швидкості кожного разу, коли він викликається), get_speed (метод повинен повернути поточну швидкість). Створіть екземпляр класу Car і викличте метод accelerate п'ять разів. Після кожного виклику методу accelerate отримайте поточну швидкість автомобіля і надрукуйте її значення. Потім викличте метод brake п'ять разів. Після кожного виклику методу brake отримайте поточну швидкість автомобіля та надрукуйте її значення.

Лістинг:

```
class Car:
    def __init__(self, make, model, year):
        self.make = make
        self.model = model
        self.year = year
        self.speed = 0

    def accelerate(self):
        self.speed += 5

    def brake(self):
        if self.speed >= 5:
            self.speed -= 5
        else:
            self.speed = 0

    def get_speed(self):
        return self.speed

if __name__ == "__main__":
    my_car = Car("Toyota", "Camry", 2022)
    for _ in range(5):
        my_car.accelerate()
```

		Гришкова Е.А.			ДУ «Житомирська політехніка». 25.F2.02.000 – Лр8	Арк.
		Желізько В.В.				3
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    print(f"Прискорення: {my_car.get_speed()}")
for _ in range(5):
    my_car.brake()
    print(f"Гальмування: {my_car.get_speed()}")

```

Результат виконання:

```

:/study/python_labs/lab-8/task3.py
Прискорення: 5
Прискорення: 10
Прискорення: 15
Прискорення: 20
Прискорення: 25
Гальмування: 20
Гальмування: 15
Гальмування: 10
Гальмування: 5
Гальмування: 0

```

Клас Car реалізує керування динамічним параметром швидкості об'єкта. Методи accelerate та brake змінюють значення атрибута speed з кроком 5 одиниць. Логіка методу brake запобігає встановленню від'ємних значень швидкості, що забезпечує фізичну коректність моделі автомобіля.

Завдання 4. Напишіть клас Dog, який має три атрибути класу: mammal (ссавець), nature (характер) і breed (порода), та два атрибути екземпляра: name (кличка) і age (вік). Створіть екземпляри трьох нових собак, кожна з яких різного віку. Визначте у класі Dog метод для виведення значень атрибутів екземпляру - імені та віку конкретної собаки. За потреби, додайте кілька інших методів, які визначають поведінку собаки (подавання голосу тощо). Напишіть кілька класів, які унаслідуються від батьківського класу Dog, що описують конкретні породи собак. Визначте для цих класів атрибути nature і breed відповідно, включіть у класи по одному методу, що визначає поведінку конкретної породи собаки. Створіть батьківський клас Pets, що створює список ваших домашніх улюбленців. У підсумку, надрукуйте інформацію про ваших домашніх тварин, на зразок, як у вихідних даних.

		Гришкова Е.А.			ДУ «Житомирська політехніка». 25.F2.02.000 – Лр8	Арк.
		Желіско В.В.				4
Змн.	Арк.	№ докум.	Підпис	Дата		

Лістинг:

```
class Dog:
    mammal = "ссавець"

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def description(self):
        return f"{self.name} має вік {self.age}"

    def speak(self, sound):
        return f"{self.name} каже {sound}"

class Labrador(Dog):
    nature = "дружній"
    breed = "Лабрадор"

    def behavior(self):
        return f"{self.name} активно махає хвостом"

class Terrier(Dog):
    nature = "енергійний"
    breed = "Тер'єр"

    def behavior(self):
        return f"{self.name} швидко бігає"

class Pets:
    def __init__(self, dogs):
        self.dogs = dogs

if __name__ == "__main__":
    my_dogs = [Labrador("Болт", 3), Terrier("Рекс", 5), Dog("Джек", 2)]
    my_pets = Pets(my_dogs)
    for dog in my_pets.dogs:
        print(dog.description())
```

Результат виконання:

```
:/study/python_labs/lab-8/task4.py
Болт має вік 3
Рекс має вік 5
Джек має вік 2
```

Базовий клас Dog містить загальні атрибути та методи, тоді як спеціалізовані класи Labrador та Terrier перевизначають специфічні властивості породи (на-

		Гришкова Е.А.			ДУ «Житомирська політехніка». 25.F2.02.000 – Лр8	Арк.
		Желізко В.В.				5
Змн.	Арк.	№ докум.	Підпис	Дата		

ture, breed). Клас Pets виступає контейнером для агрегації об'єктів різних класів, що належать до однієї ієрархії.

Завдання 5. Дано послідовність цілих чисел. Необхідно її обробити і вивести на екран суму першої п'ятірки чисел із цієї послідовності, потім суму другої п'ятірки, і т. д. Але послідовність не дається відразу загалом. З плином часу до вас надходять її послідовні частини. Наприклад, спочатку перші три елементи, потім наступні шість, потім наступні два і т. д. Реалізуйте клас Buffer, який буде накопичувати в собі елементи послідовності і виводити суму п'ятірок послідовних елементів у міру їх накопичення. Однією з вимог до класу є те, що він не повинен зберігати в собі більше елементів, ніж йому дійсно необхідно, тобто, він не повинен зберігати елементи, які вже увійшли в п'ятірку, для якої була виведена сума. Клас повинен мати наступний вигляд

```
class Buffer:
    def __init__(self):
        # конструктор без аргументів

    def add(self, *a):
        # додати наступну частину послідовності

    def get_current_part(self):
        # повернути збережені в поточний момент часу елементи
        # послідовності в порядку, в якому вони були додані
```

Зверніть увагу, що під час виконання методу add виводити суму п'ятірок може знадобитися кілька разів до тих пір, поки в буфері не залишиться менше п'яти елементів.

Лістинг:

```
class Buffer:
    def __init__(self):
        self.items = []

    def add(self, *a):
        self.items.extend(a)
        while len(self.items) >= 5:
            print(sum(self.items[:5]))
            self.items = self.items[5:]

    def get_current_part(self):
        return self.items
```

		Гришкова Е.А.			ДУ «Житомирська політехніка». 25.F2.02.000 – Лр8	Арк.
		Желізко В.В.				6
Змн.	Арк.	№ докум.	Підпис	Дата		

```
if __name__ == "__main__":
    buf = Buffer()
    buf.add(1, 2, 3)
    print(f"Залишок: {buf.get_current_part()}")
    buf.add(4, 5, 6, 7, 8, 9, 10, 11)
    print(f"Залишок: {buf.get_current_part()}")
```

Результат виконання:

```
Залишок: [1, 2, 3]
15
40
Залишок: [11]
```

Клас Buffer реалізує алгоритм накопичувальної обробки поточкових даних. Метод add використовує механізм зрізів (slicing) для виокремлення п'ятірок елементів та їхнього підсумовування. Важливою умовою є ефективне керування пам'яттю: після обробки елементи видаляються зі списку, залишаючи лише ті, що чекають на формування наступної повної групи

Завдання 6. Напишіть клас-виняток, на основі вбудованого в Python класу ValueError(). Клас буде представляти перевірку певного імені на основі його довжини. Якщо довжина введенного імені є меншою 10, то має генеруватися виняток як у вихідних даних. У інших випадках нічого не виводиться.

Лістинг:

```
class NameLengthError(ValueError):
    pass

def check_name(name):
    if len(name) < 10:
        raise NameLengthError(f"Ім'я '{name}' занадто коротке")

if __name__ == "__main__":
    try:
        check_name("Еліна")
    except NameLengthError as e:
        print(f"Помилка: {e}")
```

Результат виконання:

```
PS D:\study\python_labs> python lab-8/task6.py
Помилка: Ім'я 'Еліна' занадто коротке
PS D:\study\python_labs>
```

		Гришкова Е.А.			ДУ «Житомирська політехніка». 25.F2.02.000 – Лр8	Арк.
		Желіско В.В.				7
Змн.	Арк.	№ докум.	Підпис	Дата		

Програма демонструє створення користувацького класу винятку шляхом успадкування від вбудованого класу ValueError. Це дозволяє створювати специфічні типи помилок для валідації. Виняток генерується за допомогою оператора raise та обробляється блоком try-except.

Завдання 7. Напишіть один клас для перетворення десяткового числа на число в римській системі числення. І ще один клас для перетворення числа з римської системи числення у десяткове число.

Лістинг:

```
class ToRoman:
    def convert(self, num):
        val = [1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1]
        syb = ["M", "CM", "D", "CD", "C", "XC", "L", "XL", "X", "IX", "V", "IV", "I"]
        roman_num = ''
        i = 0
        while num > 0:
            for _ in range(num // val[i]):
                roman_num += syb[i]
                num -= val[i]
            i += 1
        return roman_num

class FromRoman:
    def convert(self, s):
        roman = {'I': 1, 'V': 5, 'X': 10, 'L': 50, 'C': 100, 'D': 500, 'M': 1000}
        int_val = 0
        for i in range(len(s)):
            if i > 0 and roman[s[i]] > roman[s[i - 1]]:
                int_val += roman[s[i]] - 2 * roman[s[i - 1]]
            else:
                int_val += roman[s[i]]
        return int_val

if __name__ == "__main__":
    print(f"1994 у римських: {ToRoman().convert(1994)}")
    print(f"MCMXCIV у десяткових: {FromRoman().convert('MCMXCIV')}")
```

Результат виконання:

```
:/study/python_labs/lab-8/task7.py
1994 у римських: MCMXCIV
MCMXCIV у десяткових: 1994
```

Реалізовано два класи для конвертації систем числення. ToRoman використовує жадібний алгоритм, віднімаючи найбільші можливі значення. FromRoman

		Гришкова Е.А.			ДУ «Житомирська політехніка». 25.F2.02.000 – Лр8	Арк.
		Желізько В.В.				8
Змн.	Арк.	№ докум.	Підпис	Дата		

базується на ітераційному аналізі рядка, де враховується правило віднімання (якщо символ із меншим значенням стоїть перед більшим).

Завдання 8. Онлайн-магазин.

Лістинг:

```
class Shop:
    def __init__(self, shop_name, store_type):
        self.shop_name = shop_name
        self.store_type = store_type
        self.number_of_units = 0

    def describe_shop(self):
        print(f"Магазин: {self.shop_name}, Тип: {self.store_type}")

    def open_shop(self):
        print(f"Онлайн-магазин {self.shop_name} тепер відкритий!")

    def set_number_of_units(self, number):
        self.number_of_units = number

    def increment_number_of_units(self, count):
        self.number_of_units += count

class Discount(Shop):
    def __init__(self, shop_name, store_type):
        super().__init__(shop_name, store_type)
        self.discount_products = []

    def get_discounts_products(self):
        print(f"Список товарів зі знижкою: {' '.join(self.discount_products)}")

if __name__ == "__main__":
    store = Shop("Glovo Market", "Продукти")
    print(f"Атрибут 1: {store.shop_name}")
    print(f"Атрибут 2: {store.store_type}")
    store.describe_shop()
    store.open_shop()

    # b
    shop1 = Shop("Comfy", "Електроніка")
    shop2 = Shop("Watsons", "Косметика")
    shop3 = Shop("Intertop", "Взуття")
    shop1.describe_shop()
    shop2.describe_shop()
    shop3.describe_shop()

    # c, d
```

		Гришкова Е.А.			ДУ «Житомирська політехніка». 25.F2.02.000 – Лр8	Арк.
		Желіско В.В.				9
Змн.	Арк.	№ докум.	Підпис	Дата		

```

print(f"Кількість видів товару: {store.number_of_units}")
store.set_number_of_units(150)
store.increment_number_of_units(25)
print(f"Оновлена кількість: {store.number_of_units}")

# e
store_discount = Discount("Cyber Week", "Розпродаж")
store_discount.discount_products = ["Смартфон", "Ноутбук", "Навушники"]
store_discount.get_discounts_products()

```

Результат виконання:

```

Атрибут 1: Glovo Market
Атрибут 2: Продукти
Магазин: Glovo Market, Тип: Продукти
Онлайн-магазин Glovo Market тепер відкритий!
Магазин: Comfy, Тип: Електроніка
Магазин: Watsons, Тип: Косметика
Магазин: Intertop, Тип: Взуття
Кількість видів товару: 0
Оновлена кількість: 175
Список товарів зі знижкою: Смартфон, Ноутбук, Навушники

```

У цій частині розроблено модель об'єкта роздрібної торгівлі. Реалізовано методи для модифікації внутрішнього стану (кількості товарів) як прямим встановленням значення (set_number_of_units), так і відносним збільшенням (increment_number_of_units). Це демонструє гнучкість управління атрибутами об'єкта.

Завдання 9. Облік користувачів на сайті.

Лістинг:

```

class User:
    def __init__(self, first_name, last_name, nickname, email):
        self.first_name = first_name
        self.last_name = last_name
        self.nickname = nickname
        self.email = email
        self.login_attempts = 0

    def describe_user(self):
        print(f"Користувач: {self.first_name} {self.last_name}")

    def greeting_user(self):
        print(f"Вітаємо у системі, {self.nickname}!")

    def increment_login_attempts(self):
        self.login_attempts += 1

```

		Гришкова Е.А.			ДУ «Житомирська політехніка». 25.F2.02.000 – Лр8	Арк.
		Желізько В.В.				10
Змн.	Арк.	№ докум.	Підпис	Дата		

```

def reset_login_attempts(self):
    self.login_attempts = 0

class Privileges:
    def __init__(self):
        self.privileges = ["Allowed to add message", "Allowed to delete users",
"Allowed to ban users"]

    def show_privileges(self):
        print(f"Доступні привілеї: {' '.join(self.privileges)}")

class Admin(User):
    def __init__(self, first_name, last_name, nickname, email):
        super().__init__(first_name, last_name, nickname, email)
        self.priv = Privileges()

if __name__ == "__main__":
    user1 = User("Еліна", "Гришкова", "vtk251gea", "vtk251gea@test.com")
    user1.describe_user()
    user1.greeting_user()

    user1.increment_login_attempts()
    user1.increment_login_attempts()
    print(f"Кількість спроб входу: {user1.login_attempts}")
    user1.reset_login_attempts()
    print(f"Спроби після скидання: {user1.login_attempts}")

    admin_user = Admin("Ольга", "Ніщенка", "super_admin", "admin@system.ua")
    admin_user.priv.show_privileges()

```

Результат виконання:

```

:/study/python_labs/lab-8/task9.py
Користувач: Еліна Гришкова
Вітаємо у системі, vtk251gea!
Кількість спроб входу: 2
Спроби після скидання: 0
Доступні привілеї: Allowed to add message, Allowed to delete users, Allowed to ban users

```

Програма моделює профіль користувача веб-ресурсу. Клас User інкапсулює персональні дані та лічильник спроб автентифікації. Методи інкременту та скидання (reset_login_attempts) дозволяють контролювати активність користувача, що є базовим елементом систем безпеки в прикладних програмах.

		Гришкова Е.А.			ДУ «Житомирська політехніка». 25.F2.02.000 – Лр8	Арк.
		Желізко В.В.				11
Змн.	Арк.	№ докум.	Підпис	Дата		

Посилання на репозиторій: <https://github.com/wghtl4s/Python>

Висновки: У ході виконання лабораторної роботи №8 було детально вивчено розширені можливості об'єктно-орієнтованого програмування в Python. Було реалізовано механізми інкапсуляції

		Гришкова Е.А.			ДУ «Житомирська політехніка». 25.F2.02.000 – Лр8	Арк.
		Желізко В.В.				12
Змн.	Арк.	№ докум.	Підпис	Дата		